DATA SCIENCE MINOR PROJECT REPORT

A Project report

Submitted in partial fulfilment of the requirements for the award of degree of B.Tech

(Computer Science & Engineering)

LOVELY PROFESSIONAL

UNIVERSITY PHAGWARA,

PUNJAB



PYTHON / DATA SCIENCE PROJECT

SMARTPHONE PRICE ANALYSIS AND PREDICTION USING MACHINE LEARNING

Submitted by

Divyanshu Yadav

Registration No.: 12406861

Programme and Section: B.Tech CSE / K3P24MR

Course Code: **INT234**

Under the Guidance of

Dr. Tanima Thakur

Discipline of CSE/IT

Lovely School of [SCHOOL NAME]

Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that Divyanshu Yadav, bearing Registration No. 12406861, student of B.Tech CSE under the CSE/IT Discipline at Lovely Professional University, has completed the project titled “Smartphone Price Analysis and Prediction Using Machine Learning” under my guidance and supervision.

To the best of my knowledge, the present project report represents the student’s work, effort, and study. The report is submitted in partial fulfilment of the requirements of the relevant course.

Lovely Professional University, Phagwara, Punjab

Date: 29/10/2026

DECLARATION

I, **Divyanshu Yadav**, student of B.Tech CSE under the CSE/IT Discipline at Lovely Professional University, Punjab, hereby declare that the project report titled “Smartphone Price Analysis and Prediction Using Machine Learning” has been prepared as part of my academic work.

The report describes the Python code, data-cleaning process, exploratory data analysis, regression models, and classification models implemented for this project. Any external datasets, libraries, and reference materials used have been acknowledged in the References section.

I understand that I am responsible for verifying the final results generated by executing the code on the actual dataset and for ensuring that the submitted report reflects the work performed.

Date: 29/10/2026

Registration No.: 12406861

Name of Student: Divyanshu Yadav

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide and faculty members for their guidance and support during the development of this Data Science minor project. Their suggestions helped me understand how data preparation, exploratory analysis, visualization, and machine-learning models can be combined to study a real-world problem.

I am also thankful to Lovely Professional University for providing an academic environment in which I could apply concepts from Python programming, statistics, data visualization, and machine learning. I acknowledge the developers and maintainers of Python and the open-source libraries used in this project, including pandas, NumPy, Matplotlib, and scikit-learn.

Finally, I would like to thank everyone who provided feedback or encouragement during the completion of this project.

ABSTRACT

Smartphones differ in price because of factors such as brand, launch year, memory capacity, display characteristics, battery capacity, camera specifications, connectivity, and operating system. This project uses Python to explore smartphone specifications and build machine-learning models that estimate price and classify phones into broad price categories.

The program reads a CSV dataset, standardizes column names, extracts numeric values from descriptive fields, converts selected yes/no specifications into binary indicators, applies basic validity checks, removes records without a valid positive price, and fills missing numeric feature values with their medians. Exploratory Data Analysis (EDA) is then performed using descriptive statistics, histograms, brand counts, year-wise counts, a correlation heatmap, and scatter plots.

For price estimation, the project compares Multiple Linear Regression with degree-two Polynomial Regression. Because the price distribution is expected to be strongly skewed, the target is transformed using the natural logarithm. For classification, price values are divided into three quantile-based categories—Budget, Mid-range, and Premium—and five algorithms are evaluated: K-Nearest Neighbours, Gaussian Naive Bayes, Decision Tree, Support Vector Machine, and Logistic Regression.

The code calculates regression metrics (MAE, MSE, RMSE, and R-squared) and classification metrics (accuracy, macro precision, macro recall, macro F1-score, log loss, and multiclass ROC-AUC). It also produces confusion matrices and model-comparison visualizations. Exact numerical findings must be inserted from the output produced by running the program against the supplied dataset; they are intentionally not fabricated in this report.

TABLE OF CONTENTS

Certificate

Declaration

Acknowledgement

Abstract

1. Introduction

2. Source of Dataset

3. EDA Process

4. Analysis on Dataset

4.1 Data Loading and Initial Inspection

4.2 Data Cleaning and Feature Preparation

4.3 Price Distribution Analysis

4.4 Brand-Wise Analysis

4.5 Year-Wise Analysis

4.6 Correlation Analysis

4.7 RAM versus Price

4.8 Battery versus Price

4.9 Regression Analysis

4.10 Classification Analysis

4.11 Model Comparison and Best Model

5. Conclusion

6. Future Scope

7. References

Appendix A: Code Workflow

Appendix B: Results to Fill After Execution

Appendix C: Glossary

Note: In Microsoft Word, update or replace this manual contents page with an automatic Table of Contents after applying heading styles.

LIST OF FIGURES

- Figure 1. Overall project workflow
- Figure 2. Histogram of smartphone prices
- Figure 3. Top ten brands by record count
- Figure 4. Number of phones by announcement year
- Figure 5. Correlation heatmap
- Figure 6. RAM versus price scatter plot
- Figure 7. Battery capacity versus price scatter plot
- Figure 8. Histogram of log-transformed prices
- Figure 9. Actual versus predicted log-price plot
- Figure 10. Confusion matrices for classification algorithms
- Figure 11. Classification metric comparison
- Figure 12. Decision tree visualization

LIST OF TABLES

- Table 1. Main project objectives
- Table 2. Dataset fields used by the code
- Table 3. Data-cleaning rules
- Table 4. Engineered features
- Table 5. Regression evaluation metrics
- Table 6. Classification evaluation metrics
- Table 7. Model comparison results
- Table 8. Limitations and proposed improvements

1. INTRODUCTION

1.1 Background

Smartphones are complex consumer products with specifications that vary widely across manufacturers and model generations. A device may have a high-resolution display, larger memory, a high-capacity battery, or advanced connectivity, and these characteristics may be associated with its market price. Brand reputation and the year of announcement can also be related to price.

A dataset containing smartphone specifications provides an opportunity to investigate these relationships systematically. Python makes it possible to clean raw text fields, summarize data, visualize patterns, and train predictive models in a reproducible workflow.

1.2 Problem Statement

The project addresses two connected problems. First, it examines how smartphone attributes vary across the dataset and how numeric attributes relate to the recorded approximate price in euros. Second, it trains machine-learning algorithms to predict a phone's price on a logarithmic scale and to classify a phone into one of three relative price groups.

1.3 Aim

The main aim is to develop a Python-based analysis pipeline that transforms smartphone specification data into understandable visual insights and evaluates regression and classification models for price-related prediction tasks.

1.4 Objectives

- Load a smartphone CSV dataset and inspect its dimensions, fields, data types, and missing values.
- Convert descriptive text values such as RAM, internal memory, display dimensions, battery capacity, and camera resolution into numeric features.
- Apply basic validity checks and handle missing values in a consistent way.
- Explore the distribution of price, brand representation, announcement years, and relationships among selected features.
- Train and compare Multiple Linear Regression and degree-two Polynomial Regression for log-price estimation.

- Train and compare KNN, Gaussian Naive Bayes, Decision Tree, SVM, and Logistic Regression for three-class price categorization.
- Evaluate models using suitable regression and classification metrics.
- Identify limitations and propose improvements for a more reliable future system.

1.5 Scope

The project focuses on the columns and transformations implemented in the submitted Python script. The analysis uses brand, announcement year, weight, display size, display resolution, RAM, internal storage, battery capacity, primary camera resolution, NFC, audio-jack availability, LTE support, operating system, and approximate price in euros. It does not use live market prices, online scraping, user reviews, or current retail availability.

1.6 Significance

The project demonstrates a complete introductory-to-intermediate data-science workflow: data ingestion, cleaning, feature engineering, EDA, supervised learning, evaluation, and visualization. It also illustrates that a dataset may support both regression and classification, depending on how the target variable is defined.

1.7 Project Workflow

Stage	Purpose	Main tools
Data loading	Read CSV and inspect data	pandas
Cleaning	Extract and validate numeric features	re, pandas, NumPy
EDA	Summarize patterns and create plots	pandas, Matplotlib
Feature engineering	Encode brand and create price classes	LabelEncoder, qcut
Regression	Estimate log(price)	LinearRegression, PolynomialFeatures
Classification	Predict price category	KNN, GaussianNB, Decision Tree, SVC, LogisticRegression
Evaluation	Compare model performance	scikit-learn metrics

Figure 1. Overall project workflow (insert a flowchart here if required by the instructor).

2. SOURCE OF DATASET

2.1 Dataset Source and File

The code reads a local CSV file named phone_data_1.csv from a Windows Downloads folder. The exact original publisher, public download URL, licence, and collection methodology are not specified in the submitted code. These details should be added after checking the original source from which the CSV was obtained.

Dataset filename used by the script: phone_data_1.csv

Local path in the current script: C:/Users/ASUS/Downloads/phone_data_1.csv

Important: A local file path identifies where the dataset was stored on the developer's computer; it is not a public dataset citation. Replace this description with the actual dataset page or repository before final submission.

2.2 Fields Referenced in the Code

Original field	Purpose in project	Derived field / use
brand	Manufacturer/brand label	brand; reduced to top 15 plus Other and label-encoded
announced	Announcement date text	year
weight_g	Phone weight	weight_g
display_size	Display description	Combined with display_resolution to extract display_inch
display_resolution	Resolution description	resolution_mp
ram	RAM text	ram_gb
internal_memory	Internal storage text	storage_gb
battery	Battery description	battery_mah
primary_camera	Primary camera description	camera_mp
nfc	NFC availability	has_nfc
audio_jack	Audio-jack availability	has_audio_jack
network_technology	Network technology	has_4g
os	Operating system	is_android
approx_price_eur	Approximate price in euros	price and price_cat

2.3 Dataset Dimensions

The program prints the dataset shape after reading the CSV. Record the actual number of rows and columns from that output.

Raw dataset rows: [FILL FROM phones.shape]

Raw dataset columns: [FILL FROM phones.shape]

Rows retained for modelling: [FILL FROM len(clean)]

2.4 Data Dictionary and Units

The script assumes that weight is expressed in grams, display size in inches, RAM and storage can be expressed in GB or MB, battery capacity is expressed in mAh, camera resolution is expressed in megapixels (MP), display resolution is expressed as width × height pixels, and price is expressed in euros (EUR). These assumptions should be checked against the dataset documentation.

2.5 Data Quality Considerations

- The CSV reader skips malformed lines using on_bad_lines='skip'; this can remove records and should be reported as a limitation.
- Column names are stripped of surrounding spaces and converted to lowercase.
- Several fields contain mixed text and numbers, so regular expressions are used to extract numeric values.
- Missing or unparseable feature values become NaN and are later median-imputed.
- Rows without a price and rows with non-positive prices are removed before modelling.

3. EDA PROCESS

3.1 Purpose of Exploratory Data Analysis

Exploratory Data Analysis is used to understand the structure, quality, distribution, and relationships in a dataset before interpreting model output. EDA can reveal skewed variables, missing values, class imbalance, unusual values, and possible associations between device specifications and price.

3.2 Initial Inspection

The script displays the first five records with head(), reports the dataset shape, calls info() to inspect data types and non-null counts, and calculates missing values by column using

isna().sum(). These checks help determine whether fields were read correctly and which columns require cleaning.

3.3 Cleaning Text into Numeric Values

Regular expressions are used because smartphone specifications are often stored as descriptive strings rather than pure numbers. For example, a RAM value such as '2 GB RAM' is converted to 2 GB, while '512 MB RAM' is converted to 512/1024 GB. Values that cannot be parsed are converted to missing values rather than causing the entire process to fail.

3.4 Handling Missing Values

Rows with unknown prices are removed because price is the target for both modelling tasks. Non-positive prices are also removed. For the numeric predictor fields, missing values are filled with the median of the cleaned data. Median imputation is relatively robust to extreme values, but it can reduce natural variation and should be interpreted as a practical baseline rather than a perfect reconstruction.

3.5 Univariate Analysis

Univariate analysis studies one variable at a time. The project summarizes price with describe() and displays a histogram. The histogram helps show whether prices are concentrated in a lower range, spread across multiple ranges, or affected by a small number of expensive devices.

3.6 Bivariate Analysis

Bivariate analysis studies relationships between two variables. The code creates scatter plots for RAM versus price and battery capacity versus price. These plots can show whether higher values tend to accompany higher prices, while also revealing overlap, outliers, and exceptions.

3.7 Multivariate Analysis

The correlation heatmap summarizes pairwise linear correlations among numeric features, binary indicators, and price. Correlation measures association, not causation. A strong correlation does not prove that a feature causes price to change, and a weak linear correlation does not rule out nonlinear relationships.

3.8 Visualizations Produced

Visualization	Question addressed	Interpretation guide

Price histogram	How are prices distributed?	Look for skewness, concentration, and extreme values.
Top-ten brand bar chart	Which brands have the most records?	Counts reflect dataset representation, not necessarily market share.
Phones per year	How many records are associated with each year?	Counts can reflect dataset coverage and historical sampling.
Correlation heatmap	Which variables have linear associations?	Positive/negative values show direction; magnitude shows strength.
RAM-price scatter plot	How does price vary with RAM?	Look for trend, spread, and exceptions.
Battery-price scatter plot	How does price vary with battery capacity?	Look for trend and overlap across price levels.
Log-price histogram	Does log transformation reduce skew?	Compare shape with the original price histogram.
Actual vs predicted plot	How close are predictions to actual log-prices?	Points near the diagonal indicate closer predictions.
Confusion matrices	Which classes are correctly or incorrectly predicted?	Diagonal cells are correct predictions.
Metric comparison chart	Which classifier performs better on selected metrics?	Compare metrics together, not accuracy alone.

4. ANALYSIS ON DATASET

4.1 Data Loading and Initial Inspection

Introduction

The first analysis stage imports the dataset and checks whether it can be used for subsequent processing. The code uses pandas read_csv() with on_bad_lines='skip', then standardizes column names.

General Description

The first five rows provide a quick preview of the data. The shape reports row and column counts, info() reports data types and non-null counts, and isna().sum() identifies missing values.

Specific Requirements, Functions and Formulas

Function / expression	**Role**
pd.read_csv(..., on_bad_lines='skip')	Loads the CSV while skipping malformed lines.
phones.columns.str.strip().str.lower()	Removes outer spaces from column names and lowercases them.
phones.head()	Shows the first five rows.
phones.shape	Returns (number of rows, number of columns).
phones.info()	Displays column types and non-null counts.
phones.isna().sum()	Counts missing values per column.

Analysis Results

Insert the actual output from the program here. Do not report guessed row counts. Discuss whether the expected columns were present, whether numeric-looking fields were stored as text, and which fields contained missing values.

Visualization

This stage does not create a dedicated plot. The printed preview and data-quality summaries act as diagnostic outputs.

4.2 Data Cleaning and Feature Preparation

Introduction

Raw specification values often combine numbers with units and descriptive words. The cleaning stage builds a new DataFrame containing consistent, model-ready fields.

General Description

A separate clean DataFrame is created so that only selected features are retained. The brand is copied, year and specification values are extracted, binary indicators are created, and the target price is assigned.

Specific Requirements, Functions and Formulas

Derived feature	Rule implemented	Unit / meaning
year	Extract first year matching 19xx or 20xx from announced	Calendar year
weight_g	Convert weight_g to numeric; invalid values become NaN	grams
display_inch	Extract number followed by 'inches' from combined display fields	inches
resolution_mp	Multiply parsed pixel width by height and divide by 1,000,000	Megapixels-equivalent pixel count
ram_gb	Extract number and GB/MB unit; MB is divided by 1024	GB
storage_gb	Extract number and GB/MB unit; MB is divided by 1024	GB
battery_mah	Extract 3–5 digit value followed by mAh	mAh
camera_mp	Extract numeric value followed by MP	MP
has_nfc	1 if NFC text contains 'Yes',	Binary indicator

	otherwise 0	
has_audio_jack	1 if audio-jack text contains 'Yes', otherwise 0	Binary indicator
has_4g	1 if network technology contains 'LTE', otherwise 0	Binary indicator
is_android	1 if OS text contains 'Android', otherwise 0	Binary indicator
price	Copy approx_price_eur	EUR

The resolution feature is calculated as width × height / 1,000,000. It is best described as a megapixel-equivalent pixel count rather than the physical screen size.

Validity Checks

The script sets year values above 2026, display sizes above 20 inches, weights above 2,000 grams, and battery capacities above 30,000 mAh to missing. These thresholds are simple plausibility filters. Their suitability depends on the dataset's context and should be reviewed if unusual but valid devices are included.

Missing-Value Strategy

Records with missing price are dropped, as are records with price less than or equal to zero. Missing values in year, weight, display size, resolution, RAM, storage, battery, and camera features are filled with the median of each column.

Analysis Results

Insert the missing-values-after-cleaning output and the final count of rows used for ML. Explain which fields needed the most imputation. The supplied script prints these values but does not contain the resulting console output.

Visualization

No plot is created specifically for cleaning. A before-and-after missing-value table is recommended for the final report.

4.3 Price Distribution Analysis

Introduction

Price distribution analysis helps establish the scale, spread, and skewness of the target variable. It also informs the choice of transformation and the interpretation of regression errors.

General Description

The code uses clean['price'].describe() to calculate count, mean, standard deviation, minimum, quartiles, and maximum. A histogram with 40 bins shows the frequency distribution of price in euros.

Specific Requirements, Functions and Formulas

For a sample of n prices, the mean is the sum of observed prices divided by n. The standard deviation measures spread around the mean. Quartiles divide the ordered values into four parts. The histogram groups prices into intervals and counts records within each interval.

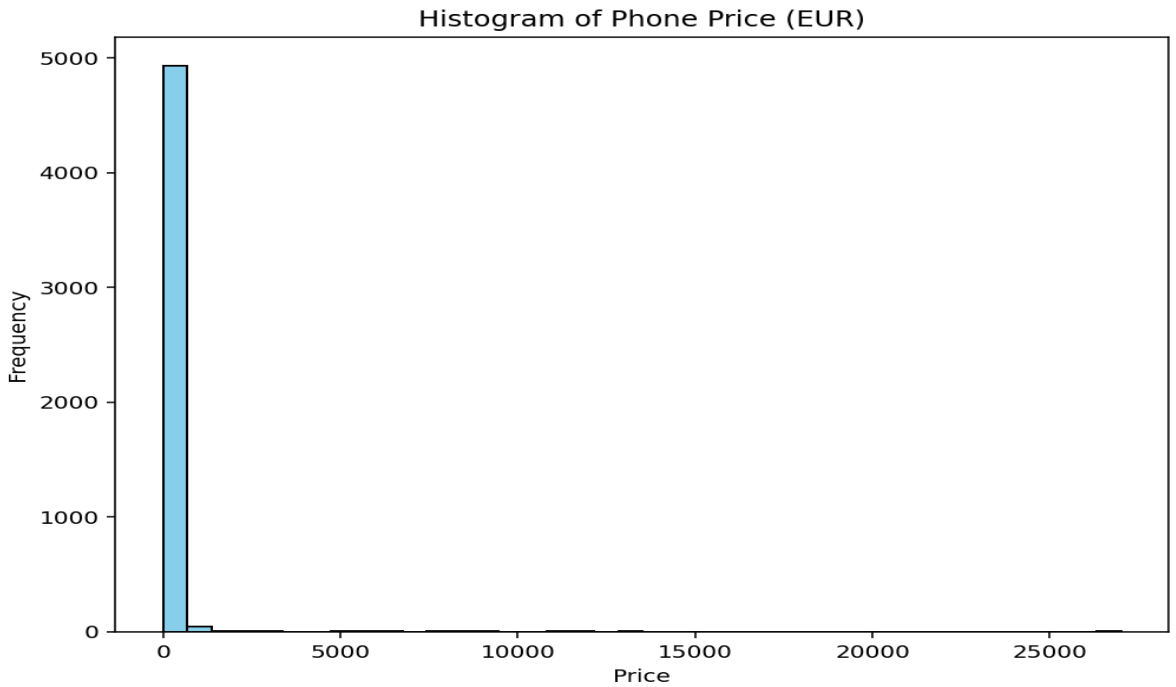
The script later uses the natural logarithm of price as the regression target: y = ln(price). This is only defined for positive values, which is why non-positive prices are removed.

Analysis Results

Report the actual count, mean, median (50th percentile), standard deviation, minimum, and maximum from the describe() output. Describe whether the histogram is right-skewed and whether the high-price tail contains relatively few observations, based on the actual plot.

Visualization

Figure 2. Histogram of Phone Price (EUR). Insert the histogram generated by the script. Figure 8 later shows the histogram after log transformation.



4.4 Brand-Wise Analysis

Introduction

Brand-wise analysis compares how many smartphone records are available for each brand in the cleaned dataset.

General Description

The code uses value_counts().head(10) to display the ten most frequently represented brands and draws a bar chart for those counts.

Specific Requirements, Functions and Formulas

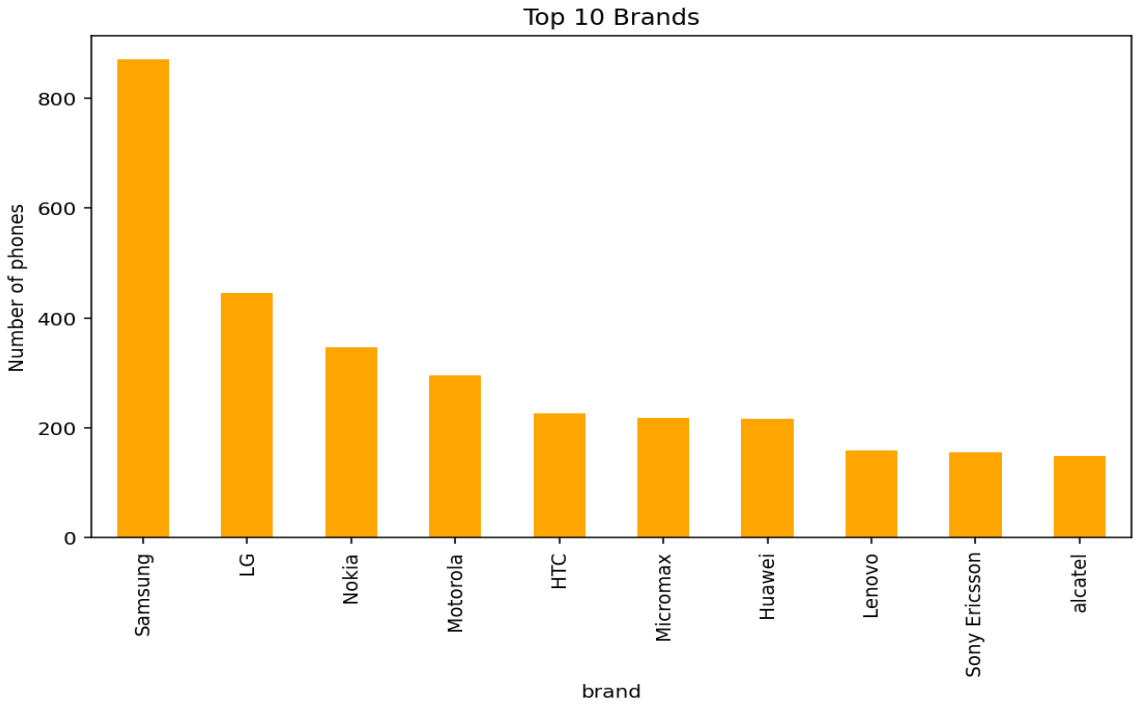
For each brand b, its record count is the number of rows whose brand label equals b. The bar chart displays the ten highest counts.

Analysis Results

Insert the printed top-ten brand table and the bar chart. State which brands have the largest counts according to the dataset. Do not describe these counts as global sales, popularity, or market share because the dataset's sampling process is not documented in the supplied code.

Visualization

Figure 3. Top 10 Brands.



4.5 Year-Wise Analysis

Introduction

The year-wise analysis examines the number of phone records associated with each extracted announcement year.

General Description

The code groups the cleaned data by year and counts rows using groupby('year').size(). It then plots the count for each year.

Specific Requirements, Functions and Formulas

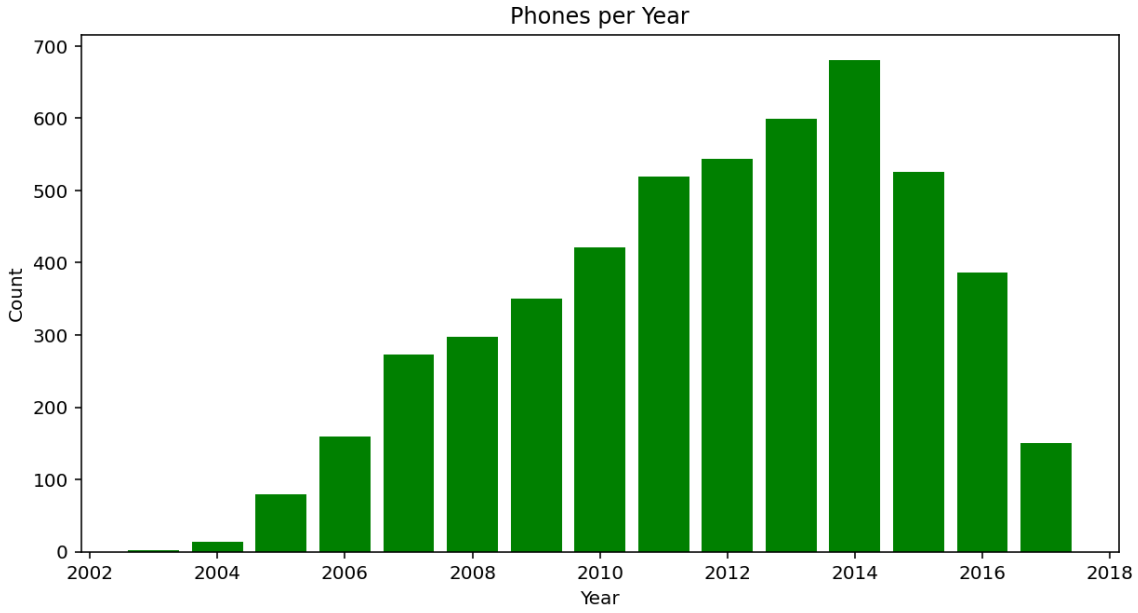
For a year t, count(t) is the number of dataset rows assigned to year t. The year is extracted from the announced text using a regular expression that recognizes years beginning with 19 or 20.

Analysis Results

Use the chart to identify years with high and low representation. Note that this is the distribution of records in the dataset, not a verified timeline of all smartphones released in those years. Median-imputed years may also affect the counts because missing year values are filled before EDA.

Visualization

Figure 4. Phones per Year.



4.6 Correlation Analysis

Introduction

Correlation analysis provides a compact view of linear relationships between numeric features and price.

General Description

The correlation matrix includes year, weight, display size, resolution, RAM, storage, battery, camera, NFC, audio jack, 4G/LTE, Android indicator, and price. The script prints the correlation of every included variable with price, sorted in descending order, and displays a heatmap.

Specific Requirements, Functions and Formulas

Pearson's correlation coefficient r measures linear association and ranges from -1 to +1. A positive value indicates that larger values of one variable tend to accompany larger values of another; a negative value indicates an inverse relationship. Values near zero indicate little linear association, not necessarily no relationship.

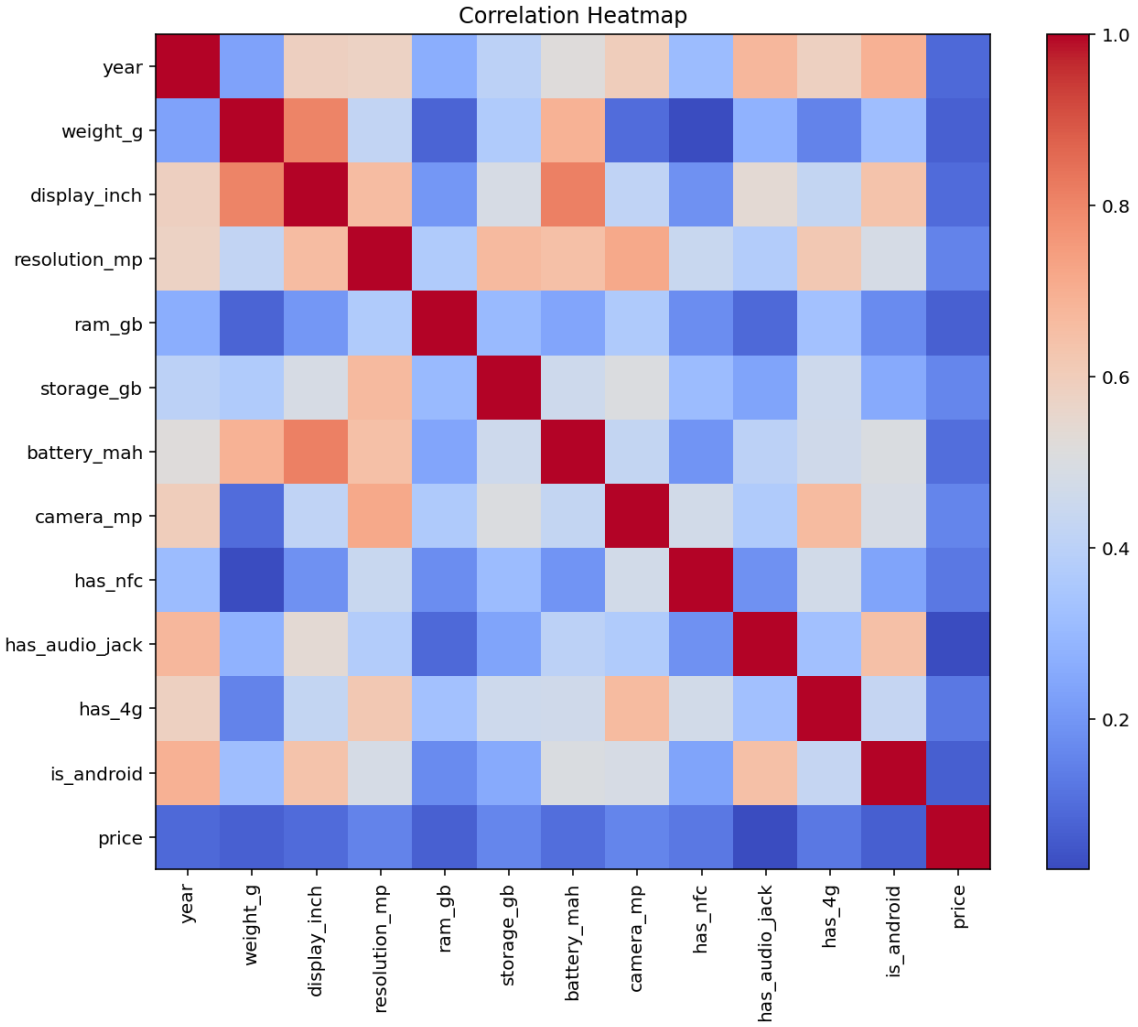
The matrix is calculated with clean[corr_cols].corr(). Binary variables are included as 0/1 indicators. Correlations may be influenced by outliers, missing-value imputation, and dependencies among features.

Analysis Results

Copy the printed 'Correlation with price' output into the report. Identify the strongest positive and negative correlations by their actual values. Avoid causal wording such as 'RAM causes higher price'; use 'is associated with' instead.

Visualization

Figure 5. Correlation Heatmap.



4.7 RAM versus Price

Introduction

RAM is a key smartphone specification that may differ substantially across budget, mid-range, and premium devices.

General Description

The script creates a scatter plot with RAM in GB on the x-axis and price in EUR on the y-axis. Transparency is used so that overlapping observations are easier to notice.

Specific Requirements, Functions and Formulas

Each point represents one cleaned smartphone record. The horizontal position is ram_gb and the vertical position is price. The chart does not fit a regression line; it is an exploratory visualization.

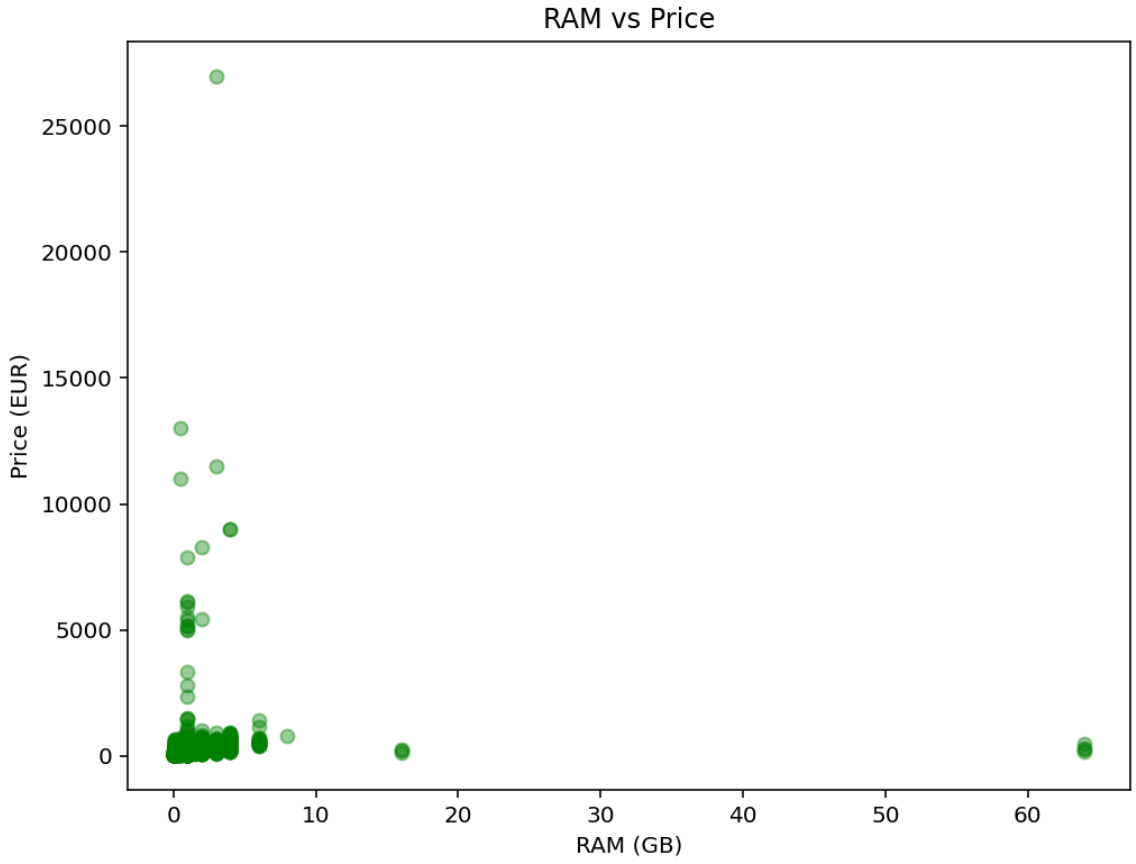
Analysis Results

Describe the visible pattern from the generated plot. Consider whether higher RAM values tend to coincide with higher prices, whether devices with the same RAM have a wide price range, and whether extreme prices are visible. The plot alone cannot establish causation.

Visualization

Figure 6. RAM versus Price.

Interpretation note: The plot contains a dense cluster of phones at relatively low prices and several extreme price observations. A few unusually large RAM values are also visible. These records should be checked against the source data before drawing firm conclusions, because data-entry or parsing issues may influence the visual pattern.



4.8 Battery Capacity versus Price

Introduction

Battery capacity is measured in milliampere-hours (mAh) and is one of several specifications that consumers may consider when choosing a smartphone.

General Description

The script plots battery_mah against price. This allows the reader to compare the price spread across phones with different battery capacities.

Specific Requirements, Functions and Formulas

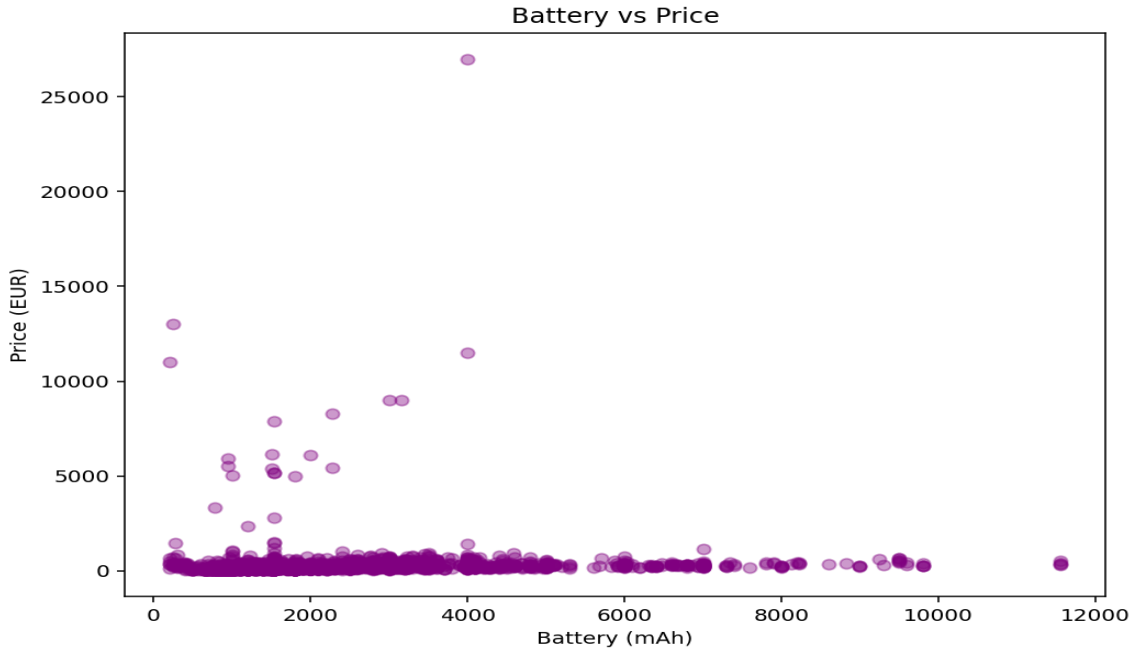
Every point represents a cleaned phone. Battery capacity is extracted from strings containing a value followed by 'mAh'. Implausibly large values above 30,000 mAh are set to missing and then median-imputed.

Analysis Results

Discuss the pattern visible in the scatter plot, including overlap between price levels and any outliers. Battery capacity alone is unlikely to represent every aspect of battery performance; efficiency, charging speed, device size, and software are not included in this plot.

Visualization

Figure 7. Battery Capacity versus Price.



4.9 Regression Analysis: Smartphone Price Estimation

Introduction

Regression predicts a continuous numeric target. In this project, regression models estimate the natural logarithm of price from smartphone specifications.

General Description

The feature matrix contains the eight numeric specification fields, four binary indicators, and encoded brand. The target is log(price). The dataset is split into training and testing subsets using an 80:20 split and random_state=123. StandardScaler is fitted on the training features and then applied to both subsets.

Model 1: Multiple Linear Regression

Multiple Linear Regression models the target as a linear combination of input features. It learns coefficients from the training set and predicts log-price for the test set.

General form: $\ln(\text{price}) = \beta 0 + \beta 1x1 + \beta 2x2 + \ldots + \beta pxp + \varepsilon$.

The implementation uses LinearRegression().fit(Xtrainscaled, ytrain). The script also prints coefficients ordered from low to high. Since the target is log-transformed, coefficients are interpreted in relation to log-price, and the effects should not be casually described as direct euro changes.

Model 2: Polynomial Regression, Degree 2

Polynomial Regression extends the linear model by creating degree-two terms from the scaled features. These terms include squared features and pairwise interactions. A linear regression model is then fitted to the expanded feature matrix.

The code uses PolynomialFeatures(degree=2, include_bias=False). This can capture some nonlinear patterns, but it increases the number of features and may overfit, especially if the dataset is small or features are highly correlated.

Training and Testing Procedure

1. Set y = log(price), where price is positive.
2. Split features and target into 80% training and 20% testing data using random_state=123.
3. Fit StandardScaler on the training features only.

4. Transform training and test features using the fitted scaler.
5. Fit each regression model on training data.
6. Predict the target for the test data.
7. Calculate MAE, MSE, RMSE, and R-squared for both models.

Evaluation Metrics

Metric	Meaning	Interpretation
MAE	Mean absolute difference between actual and predicted log-price	Lower is better; expressed in log-price units.
MSE	Mean squared prediction error	Lower is better; penalizes larger errors more strongly.
RMSE	Square root of MSE	Lower is better; in log-price units.
R-squared	Proportion of target variance explained relative to a baseline	Closer to 1 is generally better; can be negative on test data.

$$MAE = (1/n)\ \Sigma\ |yi - \hat{y}i|$$

$$MSE = (1/n)\ \Sigma\ (yi - \hat{y}i)^2$$

$$RMSE = \sqrt{MSE}$$

$$R^2 = 1 - [\Sigma(yi - \hat{y}i)^2 / \Sigma(yi - \bar{y})^2]$$

Analysis Results

Fill the table below using the console output produced by the code. The submitted source file contains the metric calculations but not their numerical output.

Regression model	MAE	MSE	RMSE	R^2
Multiple Linear Regression	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]
Polynomial Regression (degree 2)	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]

For each metric, lower MAE/MSE/RMSE is preferred; higher R^2 is generally preferred. The preferred model should be selected using the full metric pattern and an assessment of generalization, rather than a single number.

Visualization

Figure 8. Histogram of log(Price).

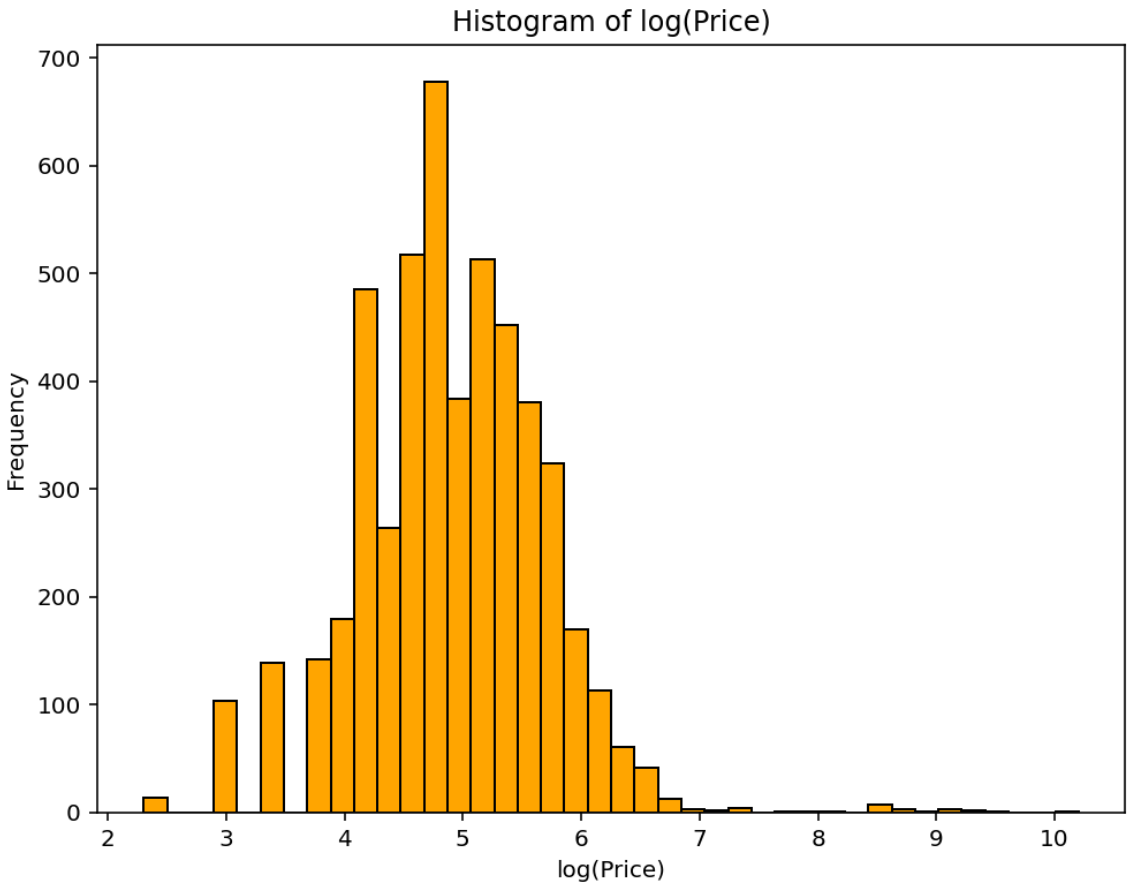
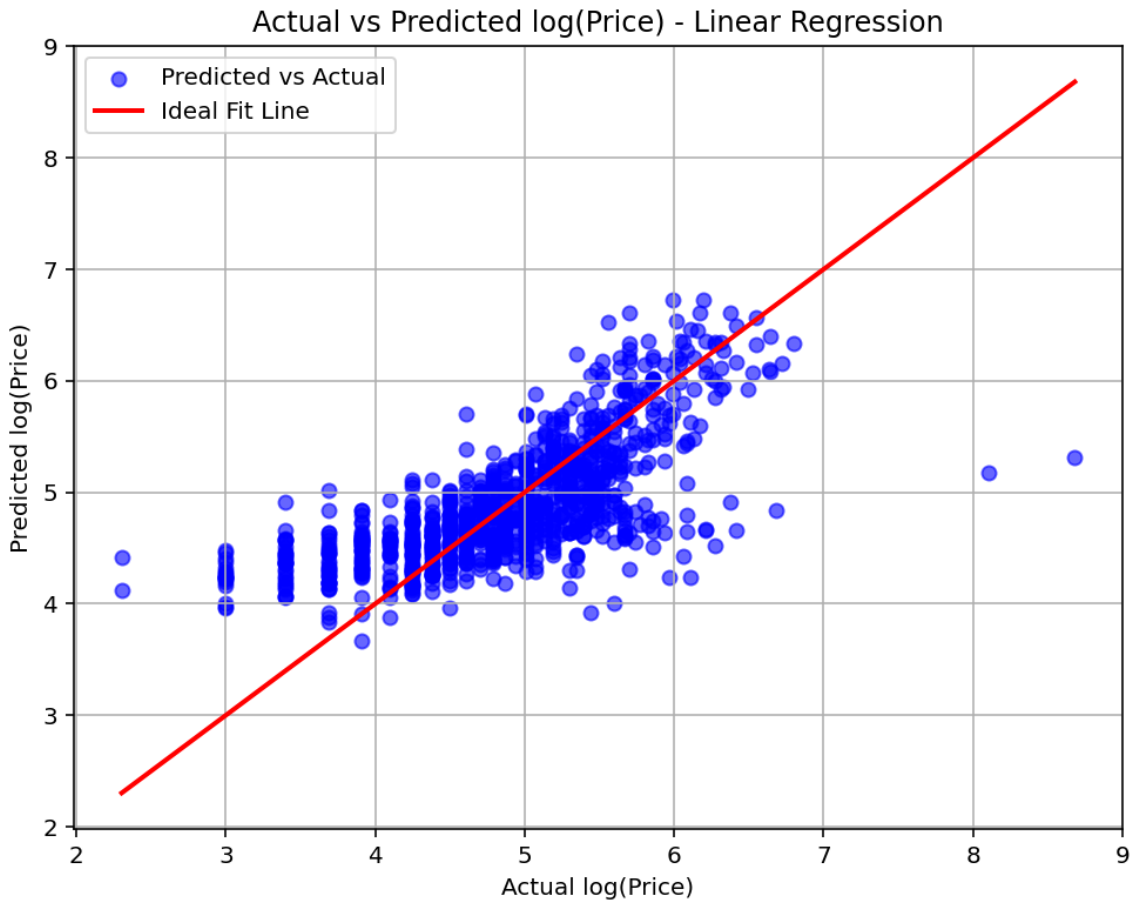


Figure 9. Actual versus Predicted log(Price) – Linear Regression.



4.10 Classification Analysis: Price Category Prediction

Introduction

Classification predicts a discrete class label. Here, the numeric price is converted into three quantile-based categories so that the models predict whether a phone belongs to a Budget, Mid-range, or Premium group.

General Description

The script creates price_cat using pandas qcut() with three quantile bins and labels 0, 1, and 2. It then performs a stratified 80:20 train-test split, scales the features, trains five classifiers, and computes multiple evaluation metrics.

Price Categories

Encoded label	Class name	How defined
0	Budget	Lowest price quantile

1	Mid-range	Middle price quantile
2	Premium	Highest price quantile

These categories are relative to the price distribution of this dataset. They are not fixed currency thresholds and should not be interpreted as universal definitions of budget, mid-range, or premium smartphones.

K-Nearest Neighbours (KNN)

KNN classifies a new observation using the labels of nearby training observations in feature space. The implementation uses n_neighbors=7. Feature scaling is important because KNN relies on distance; variables measured on larger numeric scales could otherwise dominate the distance calculation.

Gaussian Naive Bayes

Gaussian Naive Bayes applies Bayes' theorem and models each feature within each class using a Gaussian distribution. It makes a conditional-independence assumption between features given the class. This assumption is often simplified in real-world datasets, but the model can be a useful fast baseline.

Decision Tree

A Decision Tree recursively splits the feature space using rules that improve class separation. The main classifier is limited to max_depth=5 to control complexity. The script also draws a separate depth-three tree for a simpler visual explanation; that visualization model is fitted separately and is not necessarily identical to the depth-five classifier used in the comparison.

Support Vector Machine (SVM)

The SVM classifier uses an RBF kernel, which can model nonlinear decision boundaries. The code enables probability estimates so that probability-based metrics can be calculated. SVM performance can be sensitive to scaling and hyperparameter choices.

Logistic Regression

Despite its name, Logistic Regression is a classification algorithm. The implementation uses max_iter=1000 to allow additional optimization iterations. It estimates class probabilities and selects a predicted class.

Classification Evaluation Metrics

Metric	Meaning	Interpretation
Accuracy	Fraction of all predictions that are correct	Higher is better, but can hide class-specific weaknesses.
Macro precision	Average precision across classes with equal class weight	Higher means fewer false-positive assignments on average.
Macro recall	Average recall across classes with equal class weight	Higher means fewer missed instances on average.
Macro F1-score	Average F1 across classes	Balances precision and recall across classes.
Log loss	Penalty based on predicted class probabilities	Lower is better; confident wrong predictions are penalized strongly.
Multiclass ROC-AUC (OvR)	One-vs-rest area under ROC curve aggregated across classes	Higher is generally better when computed successfully.

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1 = 2 × (Precision × Recall) / (Precision + Recall)

Accuracy = Number of correct predictions / Total predictions

Confusion Matrix

A confusion matrix compares true classes with predicted classes. Rows usually represent true labels and columns represent predicted labels. Diagonal entries are correct predictions; off-diagonal entries show the categories the model confuses. The script produces a confusion matrix for every classifier.

Analysis Results

Enter the values printed in the Classification Results table after executing the script. The submitted code does not contain these output values, so this report leaves them blank rather than inventing model scores.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Log Loss	AUC
KNN	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]
Naive Bayes	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]
Decision Tree	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]
SVM	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]

Logistic Regression	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]	[RUN CODE]

Visualization

Figure 10. Confusion Matrices and Figure 11. Classification Model Comparison.

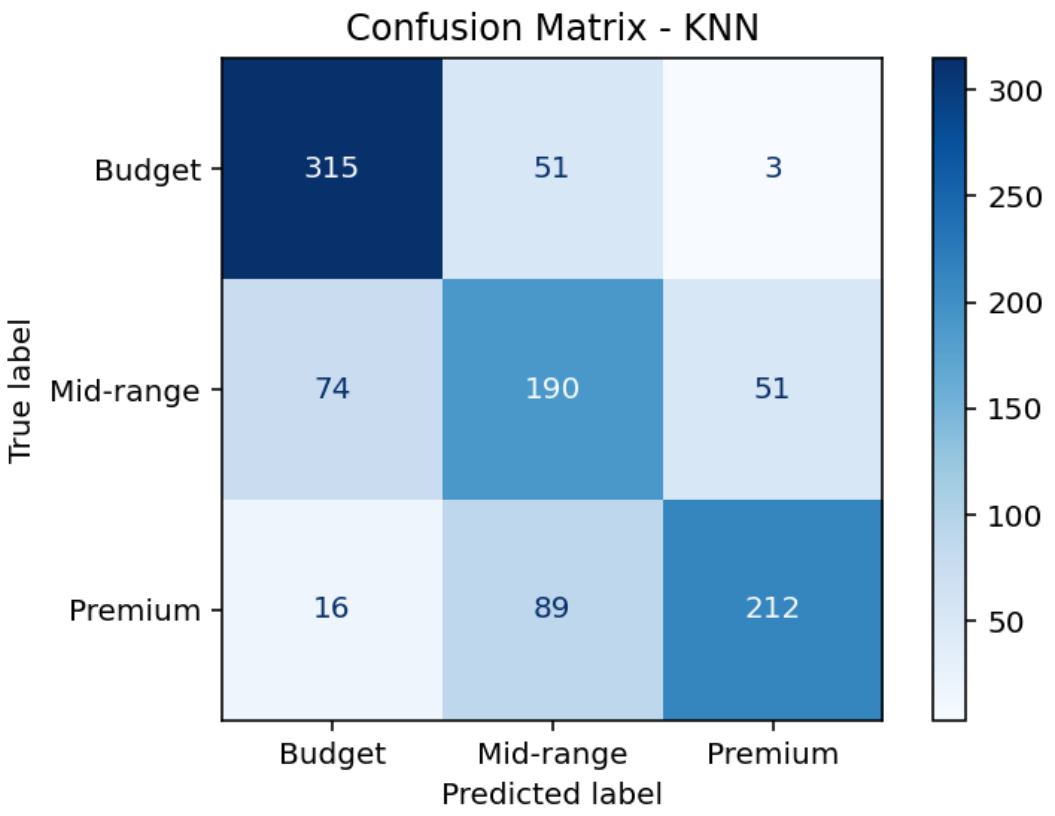


Figure 10(a). Confusion Matrix – KNN.

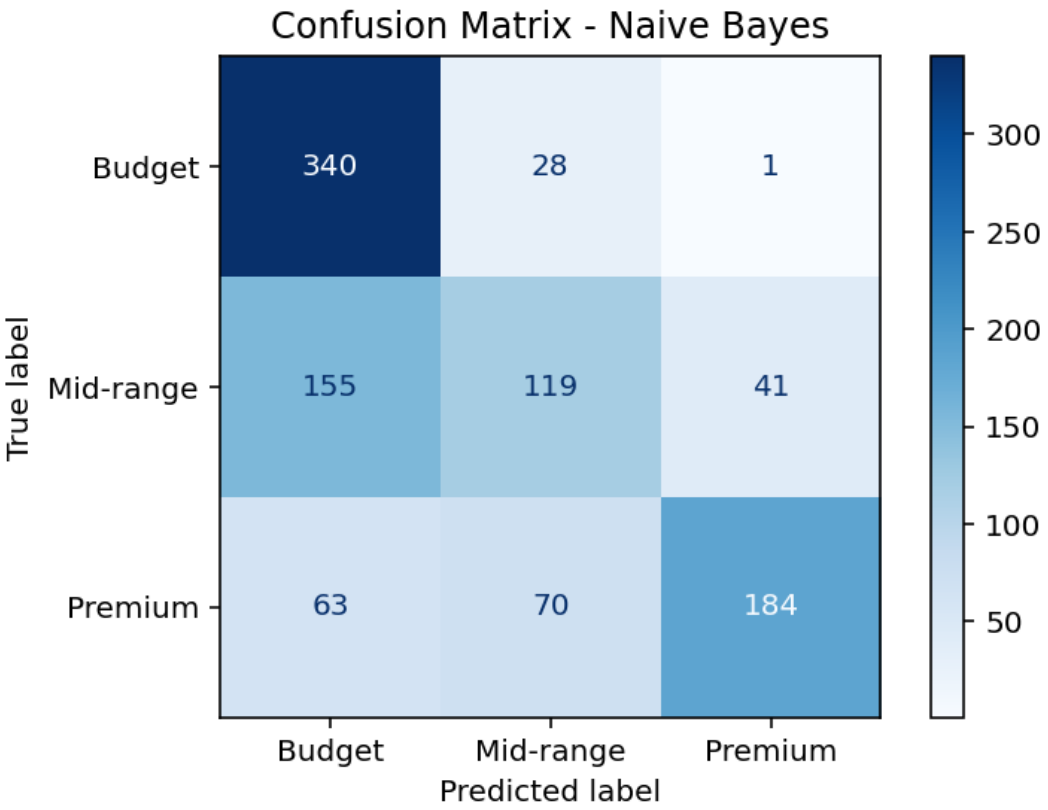


Figure 10(b). Confusion Matrix – Naive Bayes.

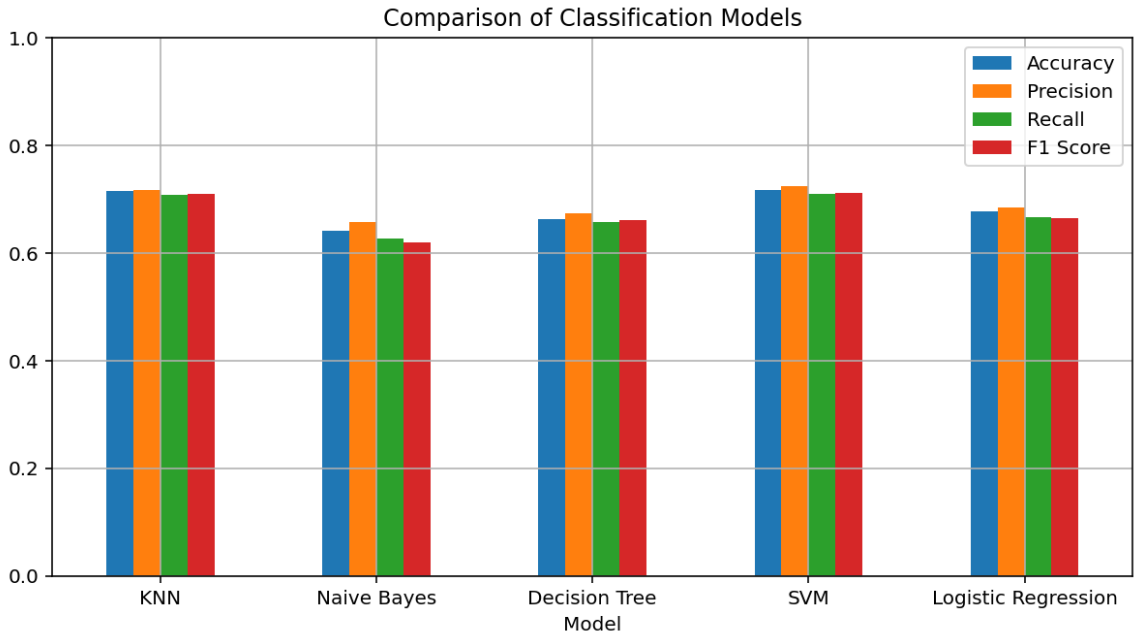


Figure 11. Comparison of Classification Models.

4.11 Model Comparison and Best Model

Introduction

Model comparison is necessary because different algorithms capture different types of patterns. A model with the highest accuracy may not have the best macro recall, probability calibration, or generalization behaviour.

General Description

The code sorts the classification results by accuracy in descending order and prints the top row as the best classification model. This is an accuracy-based selection rule.

Specific Requirements

The comparison table is rounded to three decimal places. The script uses the same stratified test split for each classifier, supporting a direct comparison on that test set. However, the code does not use cross-validation or a separate validation set for hyperparameter tuning.

Analysis Results

After running the script, report the model name and accuracy printed by the final statement. Then compare its macro precision, macro recall, macro F1, log loss, and AUC with the other classifiers. If the highest-accuracy model performs poorly on another metric, mention this trade-off.

Best classification model from program output: [FILL AFTER RUN]

Best model accuracy: [FILL AFTER RUN]

Limitations in the Current Modelling Design

- The same encoded brand values are treated as numeric values by the models, even though brand is a nominal category. One-hot encoding may be more appropriate for some algorithms.
- Median imputation is performed before the train-test split, which allows test-set feature information to influence the imputation medians. A pipeline that fits imputation only on training data would reduce leakage.

- Price categories are defined using quantiles of the complete cleaned dataset before the split. For strict evaluation, category boundaries should be learned from training data and applied to the test data.
- Scaling is applied to every classifier, which is generally useful for KNN and SVM but not essential for a Decision Tree.
- The script selects the best model using test accuracy, which can make the test set part of model selection. A validation set or cross-validation would provide a cleaner evaluation.
- The feature 'resolution_mp' is computed from the number of pixels in the display resolution; its naming should be documented carefully.
- The regex and validity thresholds may fail to capture every format in a heterogeneous smartphone dataset.

5. CONCLUSION

This project implements an end-to-end smartphone price analysis and prediction workflow in Python. The pipeline begins with CSV loading and data inspection, then converts descriptive specifications into numeric and binary features. It applies plausibility checks, removes records without valid prices, and median-imputes missing numeric predictors.

The EDA stage summarizes price, brand representation, announcement-year counts, correlations, and the relationship between price and RAM or battery capacity. The visualizations provide a practical way to inspect patterns that are difficult to identify from raw rows alone.

For continuous price estimation, Multiple Linear Regression is compared with degree-two Polynomial Regression using MAE, MSE, RMSE, and R-squared on log-price. For price-category prediction, five classifiers—KNN, Gaussian Naive Bayes, Decision Tree, SVM, and Logistic Regression—are compared using accuracy, macro precision, macro recall, macro F1-score, log loss, and multiclass ROC-AUC.

The final numerical conclusions must be based on the actual execution output. Before submission, the results tables should be completed, figures should be inserted, the original dataset source should be cited, and the report should be checked against the instructor’s requirements. No model score or dataset row count is claimed here because those outputs were not included in the supplied source code.

Key Learning Outcomes

- Using pandas and NumPy for data preparation and transformation.
- Using regular expressions to parse semi-structured product specifications.
- Applying exploratory statistics and visualizations to investigate a dataset.
- Understanding regression versus classification tasks.
- Training multiple supervised-learning algorithms using scikit-learn.
- Interpreting evaluation metrics and recognizing limitations in model comparison.

6. FUTURE SCOPE

The project can be extended in several directions to improve reliability, interpretability, and practical usefulness.

6.1 Improve Data Quality

The original dataset source, licence, date of retrieval, number of skipped malformed rows, and meaning of the price field should be documented. A more robust parser can be written for alternative display, memory, camera, and battery formats. The cleaning stage can also generate a log of rejected records and missing-value rates.

6.2 Use Leakage-Safe Preprocessing Pipelines

Imputation, scaling, and encoding should be placed inside scikit-learn Pipeline and ColumnTransformer objects. These transformations should be fitted only on training data. This makes the evaluation more realistic and reduces accidental leakage.

6.3 Improve Categorical Encoding

Brand is categorical rather than ordinal. One-hot encoding or a model with appropriate categorical handling can avoid imposing artificial numerical distances between encoded brand labels.

6.4 Use Cross-Validation and Hyperparameter Tuning

K-fold cross-validation can estimate how stable model performance is across different splits. GridSearchCV or RandomizedSearchCV can tune parameters such as the number of neighbours in KNN, tree depth, SVM C and gamma, and Logistic Regression regularization.

6.5 Add More Evaluation and Diagnostics

For regression, report errors after converting predictions back to the euro scale, with care about the statistical effect of exponentiating log predictions. Residual plots, cross-validation scores, and prediction intervals could provide further insight. For classification, report per-class precision, recall, F1-score, and normalized confusion matrices.

6.6 Explore Additional Models

Random Forest, Gradient Boosting, HistGradientBoosting, and other ensemble methods could be tested. Any improvement should be confirmed with a validation process and not judged only on one test split.

6.7 Improve Feature Engineering

Possible additions include front-camera resolution, number of camera sensors, charging speed, screen refresh rate, chipset, storage type, 5G support, launch-month features, and inflation-adjusted price. These should only be added if reliable fields exist in the dataset.

6.8 Build a User Interface

A simple Streamlit or Flask interface could allow a user to enter smartphone specifications and receive a predicted price estimate or price category. The interface should clearly state that predictions are estimates, depend on the training dataset, and are not guaranteed current retail prices.

6.9 Responsible Use

The model should not be used as a substitute for current market research. Price can vary by country, currency, condition, seller, storage configuration, taxes, and time. Predictions should be presented with uncertainty and the dataset's date and limitations.

7. REFERENCES

The following references document the Python libraries used by the supplied code. Add the actual dataset citation and access date before final submission.

8. Python Software Foundation. Python Documentation. https://docs.python.org/ (accessed: [DATE]).

9. pandas development team. pandas documentation. https://pandas.pydata.org/docs/ (accessed: [DATE]).
10. NumPy developers. NumPy documentation. https://numpy.org/doc/ (accessed: [DATE]).
11. Matplotlib Development Team. Matplotlib documentation. https://matplotlib.org/stable/ (accessed: [DATE]).
12. scikit-learn developers. scikit-learn User Guide and API Reference. https://scikit-learn.org/stable/ (accessed: [DATE]).
13. Dataset author/publisher: [ADD ORIGINAL DATASET TITLE, AUTHOR OR ORGANIZATION, URL, LICENCE, AND ACCESS DATE].

Dataset citation reminder: the filename phone_data_1.csv and local Windows path in the code do not establish the original source. Cite the web page or repository from which the dataset was downloaded.

APPENDIX A: CODE WORKFLOW

The supplied script can be understood as the following sequence:

14. Import Python libraries for regular expressions, data frames, numerical operations, plots, train-test splitting, scaling, models, and metrics.
15. Load phone_data_1.csv and normalize column names.
16. Inspect the first rows, shape, data types, and missing values.
17. Create the clean feature DataFrame and extract specification values.
18. Convert NFC, audio jack, LTE, and Android fields into binary indicators.
19. Apply plausibility thresholds and remove rows without positive prices.
20. Fill missing numeric predictors with medians.
21. Create EDA plots and print descriptive/correlation summaries.
22. Group less frequent brands into Other and label-encode the brand field.
23. Create three price classes using qcut.
24. Train and evaluate two regression models on log-price.
25. Train and evaluate five classifiers on price categories.
26. Plot confusion matrices, model metric comparisons, and a decision tree.
27. Print the classification model with the highest accuracy.

Important Reproducibility Note

The file path is specific to the original computer. To run the project on another system, change the path or place the CSV in the project directory and use a relative path. Install the required packages if they are not available in the Python environment.

Example package installation command: pip install pandas numpy matplotlib scikit-learn

APPENDIX B: RESULTS TO FILL AFTER EXECUTION

Complete this appendix using the actual console output and plots generated by the submitted script.

Item	Value to record
Raw dataset shape	[rows, columns]
Rows retained for ML	[number]
Number of brands in cleaned data	[number, if calculated]
Price mean / median / standard deviation	[values from describe()]
Top brand by record count	[brand and count]
Strongest positive correlation with price	[feature and coefficient]
Strongest negative correlation with price	[feature and coefficient]
Linear Regression MAE / MSE / RMSE / R^2	[values]
Polynomial Regression MAE / MSE / RMSE / R^2	[values]
Best classifier by accuracy	[model and accuracy]
Classification metric comparison	[paste results table]

Paste each generated plot near the corresponding analysis subsection and add a figure caption. Confirm that the plotted labels are readable in the final Word document.

APPENDIX C: GLOSSARY

Term	Definition
EDA	Exploratory Data Analysis; the process of summarizing and visualizing data.
Feature	An input variable used by a model.
Target	The output variable a model is trained to predict.
Regression	A supervised-learning task that predicts a continuous value.
Classification	A supervised-learning task that predicts a class label.
Imputation	Replacing missing values using a defined strategy.
Median	The middle value of an ordered set of observations.
Label encoding	Representing category labels with integer codes.
Feature scaling	Transforming feature values to comparable scales.
Train-test split	Separating data for model fitting and held-out evaluation.

Overfitting	When a model learns training-specific patterns that do not generalize well.
MAE	Mean Absolute Error.
MSE	Mean Squared Error.
RMSE	Root Mean Squared Error.
R-squared	A statistic comparing model residual error with target variation.
Precision	The fraction of predicted positives that are correct; generalized per class for multiclass tasks.
Recall	The fraction of actual positives that are identified.
F1-score	The harmonic mean of precision and recall.
Confusion matrix	A table comparing true and predicted class labels.
ROC-AUC	Area under the receiver operating characteristic curve.
Log loss	A metric evaluating the quality of predicted class probabilities.
Quantile	A threshold dividing ordered data into portions of a specified size.
Data leakage	When information unavailable at prediction time influences model training or evaluation.